Passerelle

Discussion de conception / Projet AWD · Éducation

Je garderais ton idée et je l'appellerais **Passerelle**. Son originalité tient à une question qu'un simple catalogue laisse ouverte pour l'étudiant : **« Avec mon parcours actuel, à quelles opportunités puis-je candidater, et que me manque-t-il pour les autres ? »**

Imagine Lina, étudiante en informatique. Son CV mentionne un projet d'analyse de données. Passerelle lui propose un stage qui demande cette compétence et lui montre quel projet justifie la suggestion. Lina trouve aussi un échange d'un semestre qui l'intéresse davantage. Son parcours correspond, mais l'université exige un justificatif d'anglais B2. Elle sait donc quelle pièce préparer avant la clôture. De son côté, le coordinateur reçoit les dossiers déposés, applique un barème annoncé et valide la liste des étudiants convoqués à l'entretien.

Je proposerais **cinq microservices métier, un par membre : Portfolio étudiant, Candidatures, Présélection, Épreuves et entretiens, et Catalogue des opportunités**. Le catalogue est le **service 5 à technologie avancée** et intègre la recommandation. **Utilisateurs et accès** est le service commun de l'équipe.

Le découpage des cinq microservices

Membre 1 — Portfolio étudiant

Sa responsabilité est de permettre à l'étudiant de constituer une seule fois le parcours qu'il réutilisera pour plusieurs candidatures.

• **Experience** → CRUD /experiences. Une expérience peut être un projet académique, un stage ou une formation. Elle possède un titre, une période, une description et un etudiantId.

• **Justificatif** → CRUD /justificatifs. Une pièce possède un type, une référence de fichier privé, une date et éventuellement une date d'expiration. Exemples : CV, relevé de notes, attestation de langue.

• **Relation locale** : une expérience peut être liée à plusieurs justificatifs. Un CV général peut aussi appartenir directement au portfolio.

• **Base proposée** : PostgreSQL pour les expériences, les métadonnées et leurs relations. Les fichiers sont stockés dans un espace privé contrôlé par le service.

Le métier que le membre peut défendre à l'oral est la **réutilisation d'un parcours qui évolue**. Si Lina améliore son CV demain, ses futures candidatures en bénéficient. Le dossier qu'elle a déposé hier conserve la version qu'elle avait choisie.

Membre 2 — Candidatures

Sa responsabilité est de transformer le portfolio en dossier destiné à **une offre précise**, puis de gérer le brouillon, le dépôt et le retrait.

• **Candidature** → CRUD /candidatures. Elle contient etudiantId, offreId, l'état, la date de dépôt et la version de l'offre utilisée.

• **PieceCandidature** → CRUD /candidatures/{id}/pieces. Chaque pièce correspond à un document choisi pour ce dossier, avec sa catégorie et sa version figée.

• **Relation locale** : une candidature contient zéro à plusieurs pièces en brouillon. Au dépôt, le service vérifie les pièces obligatoires et fige le dossier.

• **Base proposée** : PostgreSQL. Une transaction garantit que le dossier et ses pièces sont figés ensemble. Une contrainte d'unicité empêche un double dépôt du même étudiant sur la même offre.

C'est ce membre qui peut répondre à la question **« Qu'a effectivement envoyé Lina avant la date limite ? »**. Son service publie CandidatureDeposee une fois le dépôt enregistré.

Membre 3 — Présélection des dossiers

Sa responsabilité est d'aider une entreprise ou un jury universitaire à produire la **liste préliminaire des candidats à convoquer**. C'est ici que se trouve ton idée côté organisme.

• **GrillePreselection** → CRUD /grilles-preselection. L'organisme prépare un barème pour une offre, avec ses critères, ses pondérations et le nombre de personnes à convoquer.

• **EvaluationDossier** → CRUD /evaluations-dossiers. Elle contient candidatureId, la version de la grille, les points par critère, leur justification et l'avis du jury.

• **Relation locale** : une version de grille sert à évaluer plusieurs dossiers. Tous les candidats d'une campagne sont comparés avec cette même version.

• **Base proposée** : PostgreSQL pour conserver les versions de grille, les évaluations et la décision validée.

Exemple métier : pour huit places d'entretien, le service prépare un classement selon le barème publié. Si deux dossiers sont ex æquo à la huitième place, le jury voit les deux et motive son choix avant de valider la liste. Cette validation publie PreselectionValidee.

Membre 4 — Épreuves et entretiens

Sa responsabilité commence quand le jury valide la liste préliminaire. Il faut alors organiser les tests, convoquer les étudiants et enregistrer l'issue de la sélection.

• **SessionEvaluation** → CRUD /sessions-evaluation. Elle décrit un test technique, un entretien ou un oral de langue, avec date, lieu ou lien, capacité et offreId.

• **Convocation** → CRUD /convocations. Elle relie une candidature présélectionnée à une session et porte la confirmation, la présence, le résultat et, pour la dernière étape, la décision du jury.

• **Relation locale** : une session accueille plusieurs convocations. Une candidature peut passer plusieurs sessions successives.

• **Base proposée** : PostgreSQL pour réserver les places de façon transactionnelle et empêcher une double convocation à la même session.

Ce service a un métier concret même pour une seule offre : **si un créneau de six places est complet, la septième réservation doit être refusée**, y compris lorsque deux étudiants réservent en même temps.

Membre 5 — Catalogue des opportunités et compatibilité avec le parcours

Sa responsabilité est de permettre aux organismes de publier leurs opportunités et aux étudiants de découvrir celles qui leur correspondent. **La recommandation est une fonctionnalité de recherche du catalogue.** Cela donne au cinquième membre un vrai domaine métier et une technologie avancée à présenter.

• **Opportunite** → CRUD /opportunites. Elle porte le type (STAGE, ECHANGE, DOUBLE_DIPLOME, MASTER), l'organisme, le domaine, la destination, la durée, la date limite et l'état de publication.

• **CritereOffre** → CRUD /opportunites/{id}/criteres. L'organisme précise les compétences recherchées, le niveau d'études, la langue et les pièces exigées. Chaque critère indique s'il est obligatoire ou souhaité.

• **Relation locale** : une opportunité possède plusieurs critères. Une entreprise et une université utilisent le même mécanisme de publication avec des critères différents.

• **Base proposée** : PostgreSQL avec pgvector pour conserver les offres, leurs filtres structurés et les représentations vectorielles servant à la recherche sémantique.

Fonction avancée proposée : GET /opportunites/compatibles. L'étudiant choisit d'utiliser son portfolio. Le catalogue rapproche le texte de ses expériences de celui des missions proposées, puis affiche les raisons du rapprochement. Par exemple, un projet de « tableau de bord des ventes » peut être pertinent pour une offre qui parle de « visualisation de données ». Les exigences obligatoires sont vérifiées séparément. Un niveau de langue absent apparaît comme **« à justifier »**.

Vous avez ainsi **deux scores avec deux finalités précises**. Avant le dépôt, l'indice de compatibilité aide **l'étudiant** à explorer les offres. Après le dépôt, la note de présélection aide **le jury** à appliquer son barème aux dossiers figés. Le service de présélection calcule sa note à partir du dossier et de la grille : le score de découverte du catalogue n'entre pas dans ce calcul.

Service commun — Utilisateurs et accès

Toute l'équipe développe ce service et un membre le présente. **Keycloak** assure la connexion, la gestion des identifiants et l'émission des JWT. Le service commun gère les données propres à Passerelle :

• **ProfilUtilisateur** → CRUD /profils. Il relie l'identifiant Keycloak à l'identité applicative.

• **Affiliation** → CRUD /affiliations. Elle associe un profil à un organisme avec une fonction, par exemple « recruteur de l'entreprise X » ou « coordinateur de l'université Y ».

• **Relation locale** : un profil peut avoir plusieurs affiliations. Le service possède sa base PostgreSQL. Keycloak conserve son propre stockage privé.

Cette distinction répond à une vraie question d'autorisation : avoir le rôle « recruteur » permet d'accéder à l'espace entreprise, et l'affiliation détermine **quelles offres et quels dossiers** ce recruteur peut gérer.

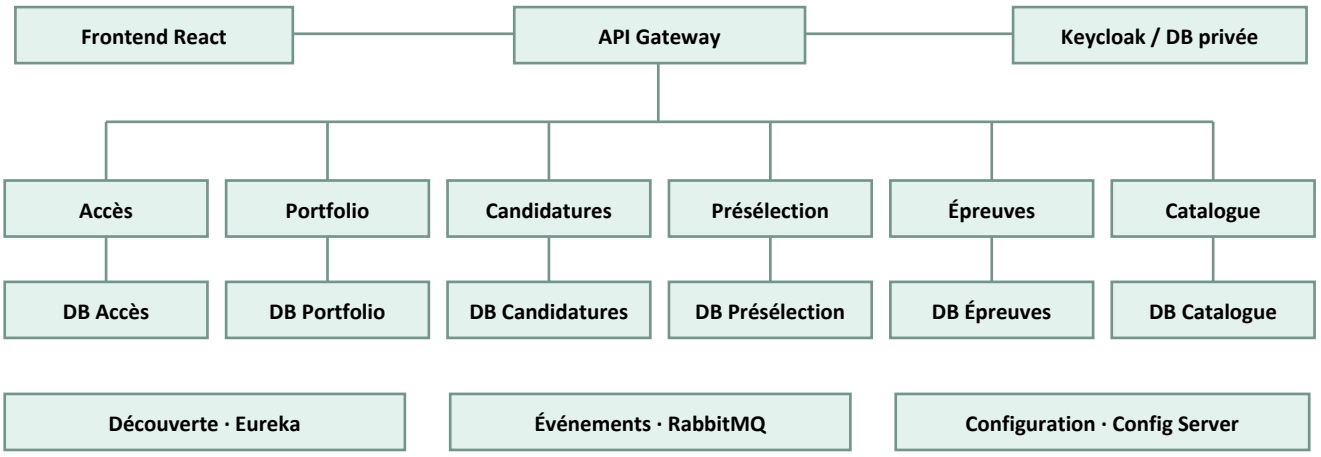
Comment les services communiquent

Interaction	Mode et données échangées	Comportement si un service est indisponible
Catalogue → Portfolio	**REST / OpenFeign** pour lire les compétences et expériences autorisées quand l'étudiant demande ses correspondances.	La recherche classique reste disponible. L'interface signale que la compatibilité personnelle est temporairement indisponible.
Candidatures → Catalogue et Portfolio	**REST / OpenFeign** au dépôt pour vérifier l'ouverture de l'offre, lire ses exigences et récupérer les versions choisies des pièces.	Le brouillon est conservé. Le dépôt est confirmé uniquement après les vérifications.
Catalogue → Utilisateurs et accès	**REST / OpenFeign** pour vérifier l'affiliation d'un recruteur avant la publication d'une offre au nom d'un organisme.	L'offre reste en brouillon jusqu'à ce que l'autorisation puisse être vérifiée.
Candidatures → Présélection	**Événement CandidatureDeposee** avec candidatureId, offreId et version du dossier. Présélection récupère ensuite le dossier figé via l'API Candidatures.	Le candidat obtient son accusé de dépôt même si Présélection est arrêté. Le traitement reprend à son retour.
Présélection → Épreuves	**Événement PreselectionValidee** avec l'offre et les identifiants des candidatures retenues.	Le traitement reprend après redémarrage. Une même validation reçue deux fois ne crée pas deux convocations.
Épreuves → Candidatures	**Événement DecisionFinaleEnregistree** pour afficher à l'étudiant la décision saisie par le jury.	La décision reste enregistrée dans Épreuves et le suivi étudiant se met à jour au retour de Candidatures.

Convention CRUD commune. Pour chacune des ressources proposées : POST crée (201), GET consulte (200), PATCH modifie (200) et DELETE supprime (204). Une ressource absente renvoie 404, un accès interdit 403, et une opération incompatible avec l'état métier 409. Par exemple, l'étudiant peut supprimer une pièce de son brouillon. Après dépôt, il peut retirer sa candidature en conservant la trace du dossier transmis.

Propriété des données. Les clés étrangères restent à l'intérieur de la base du service. Par exemple, PieceCandidature.candidatureId est une clé étrangère locale. En revanche, Candidature.offreId est un simple identifiant externe. Seul le Catalogue peut modifier l'offre.

Architecture à dessiner pour la soutenance



Je propose **Docker Compose** pour lancer l'ensemble, avec un conteneur applicatif par service. Chaque membre peut démarrer et tester son service indépendamment. Pour les événements, prévoyez une publication fiable après enregistrement en base et un identifiant d'événement permettant d'ignorer les doublons.

Ce que je présenterais en semaine 4

Périmètre proposé : modéliser les quatre types d'opportunités, puis raconter un scénario complet avec **un stage et un échange universitaire**. Cela montre ce que les entreprises et les facultés ont en commun, tout en gardant un exemple que les cinq membres peuvent expliquer en dix minutes. Chaque membre peut consacrer sa minute à une décision métier concrète : réutiliser une preuve, figer un dossier, départager un ex æquo, réserver la dernière place ou expliquer une correspondance.

Point à confirmer avec l'enseignant : la grille décrit quatre microservices plus un cinquième à technologie avancée, mais sa fiche de notation individuelle affiche quatre colonnes d'étudiants. Le document de rendu demande aussi un service commun. J'ai donc suivi ta répartition explicite de **cinq responsables métier plus le service commun**, et proposé la recherche sémantique du catalogue pour le volet avancé. [1]

Pour cette validation, l'équipe doit surtout pouvoir montrer **un projet qui compile et démarre par microservice**, un diagramme de classes et un contrat d'API par responsable, le diagramme global et les contributions Git de chacun. Le rendu prévoit aussi le dossier technique, la présentation, le planning et les risques, ainsi que le dépôt public avec l'enseignant comme collaborateur. La logique métier viendra après la validation de la conception. [2]

[1] Validation1_Grille_Evaluation.pdf [2] Rendu validation intermédiaire (semaine 4).pdf

Annexe · Demande originale

Texte de la demande, dans sa formulation d'origine.

i have a project to do it with architercture micorservice !!! and our thematque is "education" and we need to suiver les deux pdf and i have to get an idea but education is so consomee so i need to think about and idea out of the box but my brine is not brining !!! i only thoute about this idea : as internshipe is a big parte in cursuce acadimique and moste of the student are looking for mobilite a l'etranger (1 semester or 1 year extchange) or double diplome or master aabrod for alll of thores are infomation are separee u cant find it in one place and why not to help les entreprise or les auter fac with do the mecanisme to make output is liste primere of student how are accsapted to allow them to move to others tests so they will be accebted and for the student we make them a system de recomandtion based on there cv les entrprise for stage and if they want to study abrod and alll of this is with scoree befor applay to the stage for student or mobilite and after for the entreprise or fac now i want u to do le decoupage des micro service and for les service follow this and dont give me sevice avec recomandation coz ist not logive coz it can be consedrit as meiter ai in one of les service and Users + login is service en comuune and u must give me 5 outhers one for each member :

1. The big idea

- Monolith: one application, one database, everything inside.

- Microservices: many small applications, each one handles one business domain, each has its own database, and they talk to each other through APIs or messages.

Think of a shopping mall: instead of one giant shop selling everything, you have many small shops, each specialized, each with its own stock and its own staff.

2. The steps (for any project)

Step 1: List your modules and entities

Look at what you already have (your modules, classes, packages). each modules have (2 entite at lest and eache entite must have CRUD)

Step 2: Group them by business domain

Ask: "Which entities change together and belong to the same business idea?" example

- Users + login → identity domain we will do it en commun

- Products + categories → catalogue domain

- Cart + cart lines → order domain

One group = one candidate microservice. A module does not automatically become a service. Several modules can go in one service.

Step 3: Decide who owns which data

Each service owns its tables. No other service can touch them directly. If service A needs data from service B, it must ask B through its API.

Step 4: Cut the relations between services

In a monolith you have foreign keys between tables. Across services, you can't. So you replace the FK with a simple ID, and when you need details you call the other service.

Step 5: Choose how services communicate

- Synchronous (REST): A calls B and waits for the answer. Use it when you need the data right now.

- Asynchronous (events/message broker): A publishes an event ("manufacturer validated") and doesn't wait. Others react when they want. Use it for notifications, statistics, and anything that isn't urgent.

Step 6: Add the infrastructure

- API Gateway: one single entry point for the frontend, which routes requests to the right service.

- Authentication: usually JWT tokens, checked at the gateway or in each service.

- Docker + Docker Compose: each service runs in its own container.

- Message broker (optional): RabbitMQ, Kafka, or Redis.

Step 7: Redo your design documents

- One class diagram per service

- A global architecture diagram (services, gateway, databases, broker)

- Updated technologies and non-functional requirements (scalability, independence, resilience)

3. Rules to remember

1. One service = one business responsibility

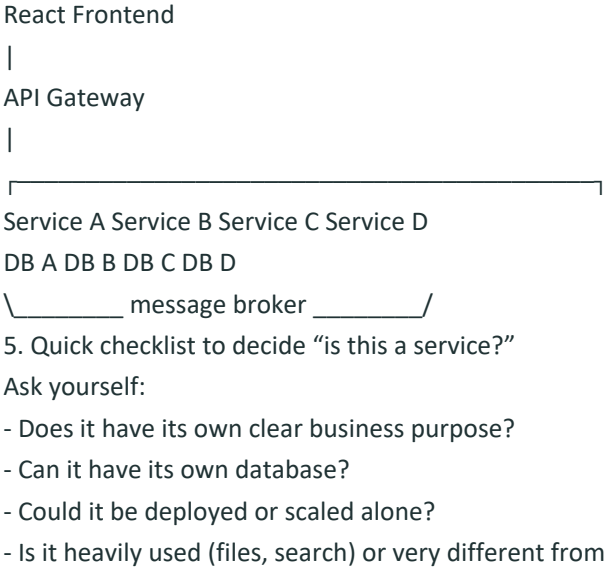
2. One database per service

3. No direct database access between services

4. Services must be independently deployable

5. Don’t over-split: start with few services, split more only if needed

4. The summary in one picture



5. Quick checklist to decide “is this a service?”

Ask yourself:

- Does it have its own clear business purpose?
- Can it have its own database?
- Could it be deployed or scaled alone?
- Is it heavily used (files, search) or very different from the rest?